

Technical Document

GameLift Dev Guide

Table of Contents:

Software:.....	2
Folder Guide:.....	2
File Naming.	3
GitHub commit Standard:.....	3
Code Standard:	4
Naming convention.	4
Spacing.....	4
Brace style.	5
Structs.	5
Flags.	6
Manager classes.	6
Declarations.	7
Classes.....	7
Comments.	7

Software:

- [Unity](https://unity.com/releases/editor/archive) of version 6000.3.24f1 . You may find and download this version here: <https://unity.com/releases/editor/archive>

Folder Guide:

- In the Unity folder you will find a [.\Build](#) and [.\GameLift](#) directories. You may add the [.\GameLift](#) directory as the new project in Unity.

The folder structure for the [.\GameLift](#) :

[.\GameLift](#)

- [.\Animations](#)
- [.\Animators](#)
- [.\Materials](#)
- [.\Models](#)
- [.\Prefabs](#)
- [.\Scenes](#)
- [.\Scripts](#)
- [.\Structs](#)
- [.\Settings](#)
- [.\Shaders](#)
- [.\Textures](#)
- [.\VFX](#)

File Naming.

- Always use Camel case (SoLikeThis)
- If you create a **material** use **MAT_** prefix.
- If you import a **model** make sure to use the right prefix:

SM_	Static
ANIM_	Animated

File extension: **.FBX**

Use **Y** up and **-Z** forward for exporting.

- If you import a texture make sure its file extension is **.PNG** or **.JPG**

GitHub commit Standard:

- All commits must add one (or multiple if they are related but don't make them big) feature which must be described in one small sentence which must be the main title of the commit.
- The description body of the commit must describe in more detail what the commit contains: adds new feature / fixes. A feature / fix must be marked with a dash ("-") at the start of each feature / fix description.
- The description must be less than 6 sentences (or bullet points).

Code Standard:

Naming convention.

- All **scripts** should start with a capital letter and use camel formatting (SoLikeThis). All the **functions** must follow this as well. No prefix or suffix.
- All non-static, private, protected or **variables** must start with a lowercase letter and follow the camel case formatting (soLikeThis).
- Other static or public global variables must start with an uppercase letter and follow the camel case formatting (SoLikeThis)
- All parameter variables in a function must start with a lowercase letter and follow the camel case formatting (soLikeThis)
- If a **function** asks for the **variable** named the same as a global variable in class in must have a “_” suffix at the end of it. **However**, avoid using the same names for variables and parameters.
- Treat abbreviations as normal words when it comes to naming convention (for example: **private string** myUrlVariable;)
- When declaring a **bool** variable try to name it as “is_” (replace “_” with the state name)

Spacing.

- Indentation should be done using **tabs** — never spaces.
- Indentation for blocks uses **1 tab** for optimal readability.

- Make sure to place one space before and one space after for each mathematical sign you use:

`(x + y; x - y; x * y; x / y;)`

Brace style.

- When creating a [function](#), the braces must be placed like this:

```
private void FunctionName (int parameterName)
{
    ...
}
```

Alternatively, if the if statement is only 1 line of code you may use this formatting (expanding braces still preferred)

```
if (true) DoSomething();
```

Structs.

- When creating specific structs to help organize the data in the Classes you must create a new .cs file (in the structs folder) following this structure:

```
[System.Serializable]
public struct StructForSomething
{
    public ...
    ...
}
```

Make one struct per file, never more.

Also don't forget to uninherit [MonoBehaviour](#) for the structs.

Flags.

- All the **variables** (if declared globally), **functions** must have an access flag (even for the built-in Unity functions like: **Start()**, **Update()** etc.)
- When making a **[SerializeField]** variable it must be in a **[Header("Category Name")]** while keeping sense with the category.

For example, a variable:

```
[SerializeField] private Vector2Int GridSize;
```

Should be in the header

```
[Header("Grid Settings")]
```

- The flags for a variable must be declared in this order:

```
[SerializeField] [Range(from, to)] private int variableName = *default value*;
```

Manager classes.

- All managing classes should have a public static instance of itself. Additionally this variable must come last and be separated with a space.

```
public static XManagerClass Instance { get; private set; }

private void Start()
{
    if (Instance != null)
    {
        Debug.Log("The Instance of XManager has been instantiated already");

        Destroy(this);
    }

    Instance = GetComponent<XManagerClass>();
}
```

Declarations.

- Don't declare more than one variable per line. Avoid doing this:

```
private int myInt, mySecondInt;
```

Classes.

- Avoid declaring more than one class per file.

Comments.

- Use comments where is necessary (whether a complicated piece of code or a note for whoever is going to be editing it). Use a space after the two backslashes like so:

```
// This is a proper comment
```

- **TODO** comments should also include a space after the slashes, followed by a colon.

```
// TODO: This is a proper todo comment
```